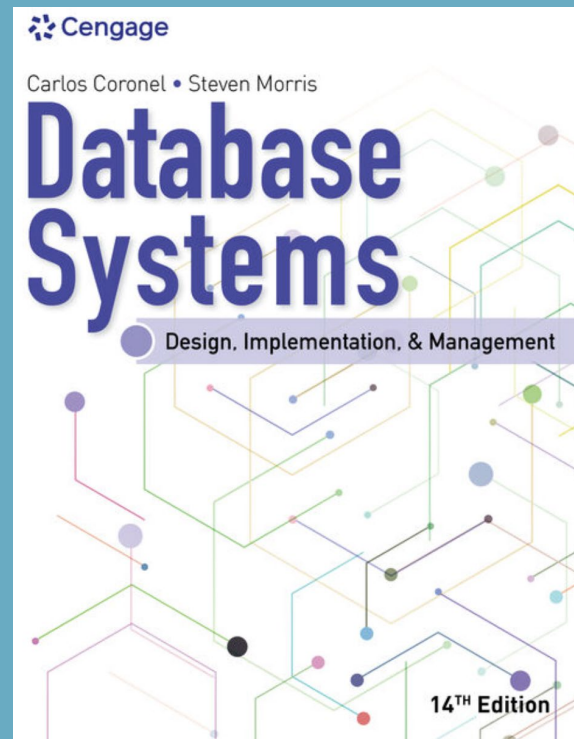




Chapter 8

Advanced SQL



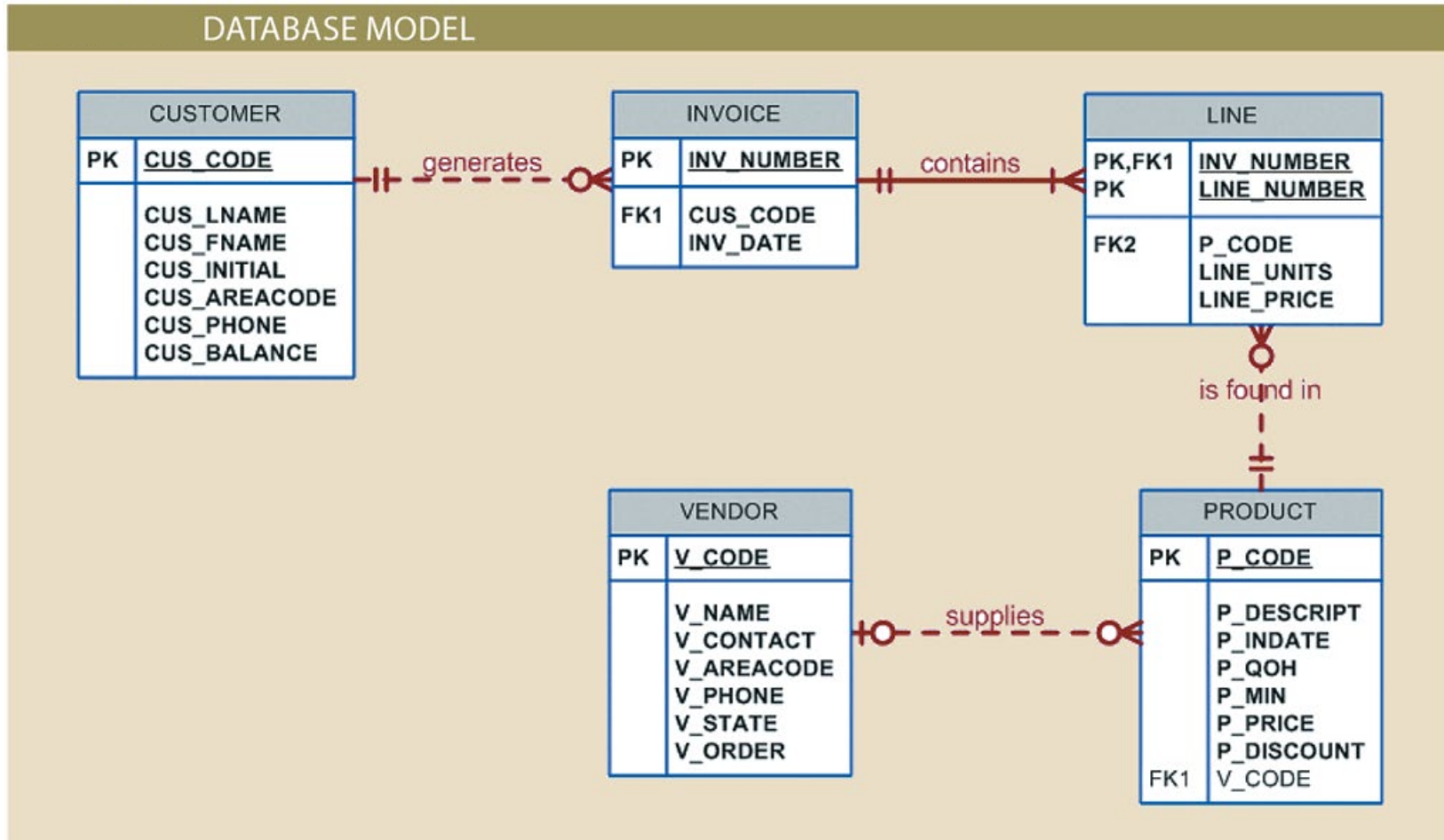
Learning Objectives

- After completing this chapter, you will be able to:
 - Use SQL to create a table manually
 - Use SQL to create a copy of a table using a subquery
 - Manipulate the structure of existing tables to add, modify, and remove columns and constraints
 - Use SQL to do data manipulation (insert, update, and delete rows of data)
 - Use SQL to create database views, including updatable views
 - Use Procedural Language SQL (PL/SQL) to create triggers, stored procedures, and PL/SQL functions
 - Create embedded SQL



Data Definition Commands (1 of 3)

- Starting database model





Data Definition Commands (2 of 3)

- Creating the database
 - Before a new database can be used, the database structure and the tables that will hold the end-user data must be created
CREATE SCHEMA
 - The DBMS system creates the physical files that will hold the data
 - Varies from system to system
- The database schema
 - Logical group of database objects—such as tables and indexes—that are related to each other
- Data types
 - Character, numeric, and date



Data Definition Commands (3 of 3)

Some Common SQL Data Types		
Data Type	Format	Comments
Numeric	NUMBER(L,D) or NUMERIC(L,D)	The declaration NUMBER(7,2) or NUMERIC(7,2) indicates that numbers will be stored with two decimal places and may be up to seven digits long, including the sign and the decimal place (for example, 12.32 or -134.99).
	INTEGER	May be abbreviated as INT. Integers are (whole) counting numbers, so they cannot be used if you want to store numbers that require decimal places.
	SMALLINT	Like INTEGER but limited to integer values up to six digits. If your integer values are relatively small, use SMALLINT instead of INT.
	DECIMAL(L,D)	Like the NUMBER specification, but the storage length is a minimum specification. That is, greater lengths are acceptable, but smaller ones are not. DECIMAL(9,2), DECIMAL(9), and DECIMAL are all acceptable.
Character	CHAR(L)	Fixed-length character data for up to 255 characters. If you store strings that are not as long as the CHAR parameter value, the remaining spaces are left unused. Therefore, if you specify CHAR(25), strings such as Smith and Katzenjammer are each stored as 25 characters. However, a U.S. area code is always three digits long, so CHAR(3) would be appropriate if you wanted to store such codes.
	VARCHAR(L) or VARCHAR2(L)	Variable-length character data. The designation VARCHAR2(25) or VARCHAR(25) will let you store characters up to 25 characters long. However, unlike CHAR, VARCHAR will not leave unused spaces. Oracle automatically converts VARCHAR to VARCHAR2.
Date	DATE	Stores dates in the Julian date format.



Creating Table Structures (1 of 3)

- CREATE TABLE command

```
CREATE TABLE tablename (  
  column1           data type           [constraint] [,  
  column2           data type           [constraint] ] [,  
  PRIMARY KEY        (column1           [, column2]) ] [,  
  FOREIGN KEY        (column1           [, column2]) REFERENCES tablename] [,  
  CONSTRAINT         constraint ] );
```

- SQL constraints

- FOREIGN KEY
- NOT NULL
- UNIQUE
- DEFAULT
- CHECK



SQL Constraints

NOT NULL

- Ensures that column does not accept nulls

UNIQUE

- Ensures that all values in column are unique

DEFAULT

- Assigns value to attribute when a new row is added to table

CHECK

- Validates data when attribute value is entered



Create Table (1 of 3)

DATA DICTIONARY FOR THE CH08_SALECO DATABASE								
TABLE NAME	ATTRIBUTE NAME	CONTENTS	TYPE	FORMAT	RANGE	REQUIRED	PK OR FK	FK REFERENCED TABLE
PRODUCT	P_CODE	Product code	VARCHAR(10)	XXXXXXXXXX	NA	Y	PK	
	P_DESCRIPT	Product description	VARCHAR(35)	XXXXXXXXXXXX	NA	Y		
	P_INDATE	Stocking date	DATE	DD-MON-YYYY	NA	Y		
	P_QOH	Units available	SMALLINT	####	0-9999	Y		
	P_MIN	Minimum units	SMALLINT	####	0-9999	Y		
	P_PRICE	Product price	NUMBER(8,2)	####.##	0.00-9999.00	Y		
	P_DISCOUNT	Discount rate	NUMBER(5,2)	0.##	0.00-0.20	Y		
	V_CODE	Vendor code	INTEGER	###	100-999		FK	VENDOR
VENDOR	V_CODE	Vendor code	INTEGER	#####	1000-9999	Y	PK	
	V_NAME	Vendor name	VARCHAR(35)	XXXXXXXXXXXXXXX	NA	Y		
	V_CONTACT	Contact person	VARCHAR(25)	XXXXXXXXXXXXXXX	NA	Y		
	V_AREACODE	Area code	CHAR(3)	999	NA	Y		
	V_PHONE	Phone number	CHAR(8)	999-9999	NA	Y		
	V_STATE	State	CHAR(2)	XX	NA	Y		
	V_ORDER	Previous order	CHAR(1)	X	Y or N	Y		

FK = Foreign key

PK = Primary key

CHAR = Fixed-length character data, 1 to 255 characters

VARCHAR = Variable-length character data, 1 to 2,000 characters. VARCHAR is automatically converted to VARCHAR2 in Oracle.

NUMBER = Numeric data. NUMBER(9,2) is used to specify numbers that have two decimal places and are up to nine digits long, including the decimal places.

Some RDBMSs permit the use of a MONEY or a CURRENCY data type.

NUMERIC = Numeric data. DBMSs that do not support the NUMBER data type typically use NUMERIC instead.

INT = Integer values only. INT is automatically converted to NUMBER in Oracle.

SMALLINT = Small integer values only. SMALLINT is automatically converted to NUMBER in Oracle.

DATE formats vary. Commonly accepted formats are DD-MON-YYYY, DD-MON-YY, MM/DD/YYYY, and MM/DD/YY.

*Not all the ranges shown here will be illustrated in this chapter. However, you can use these constraints to practice writing your own.



Create Table (2 of 3)

DATA DICTIONARY FOR THE CH08_SALECO DATABASE

TABLE NAME	ATTRIBUTE NAME	CONTENTS	TYPE	FORMAT	RANGE	REQUIRED	PK OR FK	FK REFERENCED TABLE
PRODUCT	P_CODE	Product code	VARCHAR(10)	XXXXXXXXXX	NA	Y	PK	
	P_DESCRIPTOR	Product description	VARCHAR(35)	XXXXXXXXXXXX	NA	Y		
	P_INDATE	Stocking date	DATE	DD-MON-YYYY	NA	Y		
	P_QOH	Units available	SMALLINT	####	0-9999	Y		
	P_MIN	Minimum units	SMALLINT	####	0-9999	Y		
	P_PRICE	Product price	NUMBER(8,2)	####.##	0.00-9999.00	Y		
	P_DISCOUNT	Discount rate	NUMBER(5,2)	0.##	0.00-0.20	Y		
	V_CODE	Vendor code	INTEGER	###	100-999		FK	VENDOR
VENDOR	V_CODE	Vendor code	INTEGER	#####	1000-9999	Y	PK	
	V_NAME	Vendor name	VARCHAR(35)	XXXXXXXXXXXXXX	NA	Y		
	V_CONTACT	Contact person	VARCHAR(25)	XXXXXXXXXXXXXX	NA	Y		
	V_AREACODE	Area code	CHAR(3)	999	NA	Y		
	V_PHONE	Phone number	CHAR(8)	999-9999	NA	Y		
	V_STATE	State	CHAR(2)	XX	NA	Y		
	V_ORDER	Previous order	CHAR(1)	X	Y or N	Y		

```
CREATE TABLE VENDOR (  
  V_CODE      INTEGER UNIQUE NOT NULL,  
  V_NAME      VARCHAR(35) NOT NULL,  
  V_CONTACT   VARCHAR(15) NOT NULL,  
  V_AREACODE  CHAR(3) NOT NULL,  
  V_PHONE     CHAR(8) NOT NULL,  
  V_STATE     CHAR(2) NOT NULL,  
  V_ORDER     CHAR(1) NOT NULL,  
  PRIMARY KEY (V_CODE));
```

```
CREATE TABLE VENDOR (  
  V_CODE      INTEGER PRIMARY KEY,  
  V_NAME      VARCHAR(35) NOT NULL,  
  V_CONTACT   VARCHAR(15) NOT NULL,  
  V_AREACODE  CHAR(3) NOT NULL,  
  V_PHONE     CHAR(8) NOT NULL,  
  V_STATE     CHAR(2) NOT NULL,  
  V_ORDER     CHAR(1) NOT NULL);
```



Create Table (3 of 3)

DATA DICTIONARY FOR THE CH08_SALECO DATABASE

TABLE NAME	ATTRIBUTE NAME	CONTENTS	TYPE	FORMAT	RANGE	REQUIRED	PK OR FK	FK REFERENCED TABLE
PRODUCT	P_CODE	Product code	VARCHAR(10)	XXXXXXXXXX	NA	Y	PK	
	P_DESCRIPT	Product description	VARCHAR(35)	XXXXXXXXXXXX	NA	Y		
	P_INDATE	Stocking date	DATE	DD-MON-YYYY	NA	Y		
	P_QOH	Units available	SMALLINT	####	0-9999	Y		
	P_MIN	Minimum units	SMALLINT	####	0-9999	Y		
	P_PRICE	Product price	NUMBER(8,2)	####.##	0.00-9999.00	Y		
	P_DISCOUNT	Discount rate	NUMBER(5,2)	0.##	0.00-0.20	Y		
VENDOR	V_CODE	Vendor code	INTEGER	###	100-999		FK	VENDOR
	V_CODE	Vendor code	INTEGER	#####	1000-9999	Y	PK	
	V_NAME	Vendor name	VARCHAR(35)	XXXXXXXXXXXXXX	NA	Y		
	V_CONTACT	Contact person	VARCHAR(25)	XXXXXXXXXXXXXX	NA	Y		
	V_AREACODE	Area code	CHAR(3)	999	NA	Y		
	V_PHONE	Phone number	CHAR(8)	999-9999	NA	Y		
	V_STATE	State	CHAR(2)	XX	NA	Y		
	V_ORDER	Previous order	CHAR(1)	X	Y or N	Y		

```
CREATE TABLE PRODUCT (  
  P_CODE VARCHAR(10) NOT NULL UNIQUE,  
  P_DESCRIPT VARCHAR(35) NOT NULL,  
  P_INDATE DATE NOT NULL,  
  P_QOH INTEGER NOT NULL,  
  P_MIN INTEGER NOT NULL,  
  P_PRICE DECIMAL(8,2) NOT NULL,  
  P_DISCOUNT DECIMAL(4,2) NOT NULL,  
  V_CODE INTEGER,  
  P_MIN_ORDER INTEGER,  
  P_REORDER INTEGER,  
  PRIMARY KEY (P_CODE),  
  FOREIGN KEY (V_CODE) REFERENCES VENDOR ON UPDATE CASCADE);
```



Build Out the Rest of the Model

```
CREATE TABLE CUSTOMER (  
  CUS_CODE    INTEGER PRIMARY KEY,  
  CUS_LNAME   VARCHAR(15) NOT NULL,  
  CUS_FNAME   VARCHAR(15) NOT NULL,  
  CUS_INITIAL CHAR(1),  
  CUS_AREACODE CHAR(3) DEFAULT '615' NOT NULL CHECK(CUS_AREACODE IN ('615','713','931')),  
  CUS_PHONE   CHAR(8) NOT NULL,  
  CUS_BALANCE DECIMAL(9,2) DEFAULT 0.00,  
  CONSTRAINT CUS_UI1 UNIQUE(CUS_LNAME,CUS_FNAME));
```

```
CREATE TABLE INVOICE (  
  INV_NUMBER  INTEGER PRIMARY KEY,  
  CUS_CODE    INTEGER NOT NULL,  
  INV_DATE    DATETIME DEFAULT CURRENT_TIMESTAMP NOT NULL,  
  CONSTRAINT INV_CK1 CHECK (INV_DATE > '2010-01-01'),  
  FOREIGN KEY (CUS_CODE) REFERENCES CUSTOMER(CUS_CODE));
```

```
CREATE TABLE LINE (  
  INV_NUMBER  INTEGER NOT NULL,  
  LINE_NUMBER DECIMAL(2,0) NOT NULL,  
  P_CODE      VARCHAR(10) NOT NULL,  
  LINE_UNITS  DECIMAL(9,2) DEFAULT 0.00 NOT NULL,  
  LINE_PRICE  DECIMAL(9,2) DEFAULT 0.00 NOT NULL,  
  PRIMARY KEY (INV_NUMBER, LINE_NUMBER),  
  FOREIGN KEY (INV_NUMBER) REFERENCES INVOICE(INV_NUMBER) ON DELETE CASCADE,  
  FOREIGN KEY (P_CODE) REFERENCES PRODUCT(P_CODE),  
  CONSTRAINT LINE_UI1 UNIQUE(INV_NUMBER, P_CODE));
```



Creating Table Structures (2 of 3)

- Create a table with a SELECT statement
 - Rapidly creates a new table based on selected columns and rows of an existing table using a subquery
 - Automatically copies all of the data rows returned

```
CREATE TABLE SMALL_PRODUCT_TABLE AS
SELECT
    P_CODE AS PART_CODE,
    P_DESCRIPT AS PART_DESCRIPT,
    P_PRICE AS PART_PRICE,
    V_CODE
FROM PRODUCT;
```



Creating Table Structures (3 of 3)

- When a primary key is declared, the DBMS automatically creates a unique index
`CREATE [UNIQUE] INDEX indexname ON tablename (col1 [, col2...])`
- The **CREATE INDEX** command can be used to create an index on the basis of any selected attribute(s)
- **UNIQUE** qualifier prevents a value that has been used before **from repeating**
 - Composite indexes prevent data duplication
- To delete an index use the **DROP INDEX** command



Add Some Indices

-- Q: Create index on P_INDATE

```
CREATE INDEX P_INDATEX ON PRODUCT(P_INDATE);
```

-- Q: Create composite index on V_CODE and P_CODE

```
CREATE INDEX VENPRODX ON PRODUCT(V_CODE,P_CODE);
```

-- Q: Create index on P_PRICE in descending order

```
CREATE INDEX PROD_PRICEX ON PRODUCT(P_PRICE DESC);
```

-- Q: Delete the PROD_PRICEX index

```
DROP INDEX PROD_PRICEX ON PRODUCT;
```



Altering Table Structures (1 of 4)

- All changes in the table structure are made by using the ALTER TABLE command followed by a keyword that produces the specific change you want to make
 - ADD, MODIFY, and DROP
- Changing a column's data type
 - ALTER
- Changing a column's data characteristics
 - If the column to be changed already contains data, you can make changes in the column's characteristics if those changes do not alter the data type
- Adding a column
 - You can alter an existing table by adding one or more columns
 - Be careful not to include the NOT NULL clause for the new column



Altering Table Structures (2 of 3)

- Altering a field's characteristics

Data Type:

```
ALTER TABLE tablename MODIFY (columnname(datatype));
```

Data Characteristic:

```
ALTER TABLE tablename MODIFY (columnname(characteristic));
```

- Adding primary key, foreign key, and check constraints

- Primary key syntax:

```
ALTER TABLE PART  
ADD PRIMARY KEY (PART_CODE);
```

- Foreign key syntax:

```
ALTER TABLE PART  
ADD FOREIGN KEY (V_CODE) REFERENCES  
VENDOR(V_CODE);
```

- Check constraint syntax:

```
ALTER TABLE PART  
ADD CHECK (PART_PRICE >= 0);
```



Altering Table Structures (3 of 4)

- Changing a Column's Data type

```
ALTER TABLE PRODUCT MODIFY P_DESCRIPT CHAR(50);
```

- Changing a Column's Data Characteristics

```
ALTER TABLE PRODUCT MODIFY P_PRICE DECIMAL(9,2);
```

- Adding a Column

```
ALTER TABLE PRODUCT ADD (P_SALECODE CHAR(1));
```



Altering Table Structures (4 of 4)

- Dropping a column

- Syntax:

```
ALTER TABLE VENDOR  
    DROP COLUMN V_ORDER;
```

- Deleting a table from the database

- Syntax:

```
DROP TABLE PART;
```



Data Manipulation Commands (1 of 6)

- Adding table rows
 - INSERT command syntax:
`INSERT INTO tablename VALUES (value1, value2, ..., valuen)`
 - Inserting rows with null attributes: use NULL entry
 - Inserting rows with optional attributes: indicate attributes that have required values



Add Some Vendor Data

```
INSERT INTO VENDOR  
VALUES (21225,'Bryson, Inc.','Smithson','615','223-3234','TN','Y');  
COMMIT;
```

```
INSERT INTO VENDOR  
VALUES (21226,'Superloo, Inc.','Flushing','904','215-8995','FL','N');
```



Add Some Product Data

```
INSERT INTO PRODUCT
VALUES ('11QER/31','Power painter, 15 psi., 3-nozzle','2021-03-15',8,5,109.99,0.00,21225,0,0);
```

```
INSERT INTO PRODUCT
VALUES ('13-Q2/P2','7.25-in. pwr. saw blade','2022-12-13',32,15,14.99,0.05,21225,0,0);
```

-- Insert with Null

```
INSERT INTO PRODUCT
VALUES ('BRT-345','Titanium drill bit','2020-10-17', 75, 10, 4.50, 0.06, NULL,0,0);
```

-- Insert with optional fields

```
INSERT INTO PRODUCT(P_CODE, P_DESCRIPT)
VALUES ('BRT-346',Aluminum drill bit');
```



Data Manipulation Commands (2 of 6)

- Inserting table rows with a SELECT subquery
 - Add multiple rows to a table, using another table as the source, at the same time
 - SELECT syntax:

```
INSERT INTO      target_tablename[(target_columnlist)]  
SELECT          source_columnlist  
FROM            source_tablename;
```

INSERT INTO PART

```
(PART_CODE, PART_DESCRIPT, PART_PRICE, V_CODE)  
  SELECT P_CODE, P_DESCRIPT, P_PRICE, V_CODE  
  FROM PRODUCT;
```

CREATE TABLE PART AS SELECT

```
P_CODE AS PART_CODE,  
P_DESCRIPT AS PART_DESCRIPT,  
P_PRICE AS PART_PRICE,  
V_CODE FROM PRODUCT;
```



Data Manipulation Commands (3 of 6)

- What if you just need to change existing data?
- Updating table rows
 - UPDATE command is used to modify data in a table
 - UPDATE syntax:

```
UPDATE          tablename
SET             columnname = expression [, columnname =
               expression]
[WHERE          conditionlist ];
```



Update Product Data

UPDATE PRODUCT

SET P_INDATE = '2016-01-18'

WHERE P_CODE = '13-Q2/P2';

UPDATE PRODUCT

SET P_INDATE = '2016-01-18',

P_PRICE = 16.99,

P_MIN = 10

WHERE P_CODE = '13-Q2/P2';



Data Manipulation Commands (5 of 6)

- But what do you do when you need to coordinate the update of multiple tables or records?
 - Create a transaction
- MySQL auto commits AFTER every statement by default
- BEGIN [WORK] command before SQL requires
BEGIN [WORK];
- COMMIT command syntax:
COMMIT [WORK];
- Restoring table contents
 - ROLLBACK [WORK] command is used restore the database to its previous condition
ROLLBACK [WORK];



Restore Product Data

BEGIN;

UPDATE PRODUCT

SET P_INDATE = '2022-01-18'

WHERE P_CODE = '13-Q2/P2';

UPDATE PRODUCT

SET P_INDATE = '2022-01-18',

P_PRICE = 21.99,

P_MIN = 25

WHERE P_CODE = '13-Q2/P2';

SELECT * FROM PRODUCT

ROLLBACK;

SELECT * FROM PRODUCT;



Data Manipulation Commands (6 of 6)

- Deleting table rows
 - DELETE statement syntax:

```
DELETE FROM      tablename  
[WHERE          conditionlist ];
```



Delete Product Data

```
DELETE FROM PRODUCT  
WHERE P_CODE = '13-Q2/P2';
```

```
DELETE FROM PRODUCT  
WHERE P_MIN = 5;
```

```
DELETE FROM PRODUCT;
```

FYI...by default MySQL is set for “safe mode” for updates and deletes. this means that users can-not update or delete rows from a table unless the UPDATE or DELETE command includes a WHERE clause that provides a value for the primary key.



Creating a View

- View: virtual table based on a SELECT query
 - Base tables: tables on which the view is based
- CREATE VIEW statement: data definition command that stores the subquery specification in the data dictionary
 - CREATE VIEW command syntax:

`CREATE VIEW viewname AS SELECT query`

```
CREATE VIEW PROD_STATS AS
SELECT      V_CODE, SUM(P_QOH*P_PRICE) AS TOTCOST, MAX(P_QOH) AS
            MAXQTY, MIN(P_QOH) AS MINQTY, AVG(P_QOH) AS AVGQTY
FROM        PRODUCT
GROUP BY    V_CODE;
```



Updatable Views

- Used to update attributes
 - **Batch update routine:** pools multiple transactions into a single batch to update a master table field in a single operation
- Updatable view restrictions
 - GROUP BY expressions or aggregate functions cannot be used
 - Set operators cannot be used
 - Most restrictions are based on the use of JOINS or group operators in views
- <https://dev.mysql.com/doc/refman/8.0/en/view-updatability.html>



Sequences (1 of 2)

- Many similarities in the use of sequences across these DBMSs
 - Independent object in the database
 - Have a name and can be used anywhere a value expected
 - Not tied to a table or column
 - Generate a numeric value that can be assigned to any column in any table
 - Table attribute with an assigned value can be edited and modified

```
CREATE SEQUENCE name [START WITH n] [INCREMENT BY n] [CACHE |  
NOCACHE]
```

```
INSERT INTO CUSTOMER  
VALUES (NEXT VALUE FOR CUS_CODE_SEQ, 'Walker', 'James', NULL,  
'615', '898-2007', 0.00);
```



Sequences (2 of 2)

ORACLE SEQUENCE

```
SQL Plus

SQL> CREATE SEQUENCE CUS_CODE_SEQ START WITH 20010 NOCACHE;
Sequence created.

SQL> CREATE SEQUENCE INV_NUMBER_SEQ START WITH 4010 NOCACHE;
Sequence created.

SQL> SELECT * FROM USER_SEQUENCES;

SEQUENCE_NAME      MIN_VALUE  MAX_VALUE  INCREMENT_BY  C  O  CACHE_SIZE  LAST_NUMBER  PARTITION_COUNT  S  K
-----
CUS_CODE_SEQ        1 1.0000E+28      1 N N          0      20010              N  N
INV_NUMBER_SEQ      1 1.0000E+28      1 N N          0       4010              N  N

SQL> _
```



Autoincrement

- MySQL does not support independent sequences
 - Instead, it uses the AUTO_INCREMENT attribute of an integer field
- Only 1 table field can have the auto_increment property
- That field MUST be the primary key!
- Increment is always 1 and always increases
- Can set the initial value: AUTO_INCREMENT=1000

```
CREATE TABLE EMPLOYEES (  
    EMP_NUM INT AUTO_INCREMENT PRIMARY KEY,  
    FIRST_NAME VARCHAR(50),  
    LAST_NAME VARCHAR(50),  
    MY_COUNTER INTEGER DEFAULT 1  
);
```



Procedural SQL (1 of 2)

- Procedural Language SQL (PL/SQL)
 - Use and storage of procedural code and SQL statements within the database
 - Merging of SQL and traditional programming constructs
- Procedural code is executed as a unit by DBMS when invoked by end user
 - Anonymous PL/SQL blocks
 - Triggers
 - Stored procedures
 - PL/SQL functions



Procedural SQL (3 of 3)

PL/SQL BASIC DATA TYPES	
DATA TYPE	DESCRIPTION
CHAR	Character values of a fixed length; for example: W_ZIP CHAR(5)
VARCHAR2	Variable-length character values; for example: W_FNAME VARCHAR2(15)
NUMBER	Numeric values; for example: W_PRICE NUMBER(6,2)
DATE	Date values; for example: W_EMP_DOB DATE
%TYPE	Inherits the data type from a variable that you declared previously or from an attribute of a database table; for example: W_PRICE PRODUCT.P_PRICE%TYPE Assigns W_PRICE the same data type as the P_PRICE column in the PRODUCT table



Simple Stored Procedures

- A **stored procedure** is a named collection of procedural and SQL statements
 - They are stored in the database
 - A major advantage of stored procedures is that they can be used to encapsulate and represent business transactions
- Using stored procedures offers the following advantages:
 - Stored procedures substantially reduce network traffic and increase performance
 - Stored procedures help reduce code duplication by means of code isolation and code sharing



```
CREATE PROCEDURE procedure_name ([IN | OUT | INOUT] argument  
data-type, ...)
```

```
BEGIN
```

```
[DECLARE variable_name data type [DEFAULT initial_value] ]
```

```
[BEGIN]
```

```
    Procedural SQL and SQL statements;
```

```
[END;]
```

```
END;
```



- argument specifies the parameters that are passed to the stored procedure. A stored procedure could have zero or more arguments or parameters.
- IN | OUT | INOUT indicates whether the parameter is for input, output, or both.
- data-type is one of the procedural SQL data types used in the RDBMS. The data types normally match those used in the RDBMS table creation statement.
- The body of the stored procedure must be inside a BEGIN...END; block. Within that block can be one or more DECLARE statements and other blocks of executable code.
- Variables can be declared in a DECLARE statement. You must specify the variable name, its data type, and (optionally) an initial value.



- All MySQL programs must be stored.
- MySQL expects a program to be a statement.
- In SQL, statements are delimited by a semicolon, which means the end of a statement is marked using a semicolon.
- Because this program will be a compound statement made of many other statements, a different delimiter must be used to mark the end of the collection of statements that constitute the program.
- The first line changes the statement delimiter for the interactive session
- It is common to use \$\$
- The delimiter MUST be reset to ;



```
1  delimiter $$
2  •  DROP PROCEDURE IF EXISTS TEMP_PROC $$
3  •  CREATE PROCEDURE TEMP_PROC()
4  •  BEGIN
5      INSERT INTO VENDOR VALUES (25678, 'Microsoft Corp.', 'Bill Gates', '765', '546-8484', 'WA', 'N');
6  •  END;
7  $$
8  delimiter ;
9
10 •  call temp_proc();
```



- Suppose that you want to display a more specific message after a procedure is completed
- Because stored procedures are meant to run inside the database, they are not designed to interact directly with the users.
- They are meant to be called by other programs and to interact with them in a programmatic fashion.
- As a result, there is little support for displaying information messages within a stored procedure. You can mimic displaying an information message through the use of a SELECT query.
- Notice that a SELECT query was used to send the results to the screen. Because this query returns only a literal, the SELECT query does not include a FROM clause!



```
1  delimiter $$
2  •  DROP PROCEDURE IF EXISTS TEMP_PROC$$
3  •  CREATE PROCEDURE TEMP_PROC()
4  •  BEGIN
5      INSERT INTO VENDOR VALUES (25678, 'Microsoft Corp.', 'Bill Gates', '765', '546-8484', 'WA', 'N');
6      SELECT 'New Vendor Added!' as 'Result';
7  END;
8  $$
9  delimiter ;
10
11 •  call temp_proc()
12
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

Result
New Vendor Added!



Variables

- MySQL variables are either local variables or user-defined variables.
- A user-defined variable is indicated by placing “@” at the beginning of the variable name.
- User-defined variables persist throughout a session, allowing them to persist values across programs.
- Local variables are used within a program and are lost when the program execution ends.
- MySQL local variables are declared inside the executable BEGIN...END block and should be the first part of the executable block.
- Each variable is declared separately.
- A variable can be initialized to a starting value using the DEFAULT keyword.
- If a starting value is not specified, the variable begins with a NULL value.



```
1 • DROP PROCEDURE IF EXISTS temp_proc;
2
3 delimiter $$
4
5 • CREATE PROCEDURE TEMP_PROC()
6 BEGIN
7     DECLARE MYNUM1 NUMERIC(1) DEFAULT 5;
8     DECLARE MYNUM2 NUMERIC(1);
9     DECLARE TOTAL INT;
10    DECLARE MESSAGE VARCHAR(15) DEFAULT 'The total is ';
11
12    SET MYNUM2 = 6;
13    SET TOTAL = MYNUM1 * MYNUM2;
14    SELECT CONCAT(MESSAGE, TOTAL) AS 'Result';
15
16 END;
17 $$
18
19 delimiter ;
20
21 • call temp_proc();
22
```

Result Grid	Filter Rows:	Exports:	Wrap Cell Contents:
Result			
▶ The total is 30			



Conditional Execution

- Conditional execution of code can be performed using IF statements in MySQL.
- An IF structure is composed of one or more sections of conditional code.
- It is possible to have more than two sections in a single IF structure.
- The keyword **ELSEIF** (notice that it is a single word) can be used to create IF structures with as many logical, conditional sections as needed.



```
2 delimiter $$
3 • CREATE PROCEDURE TEMP_PROC()
4 BEGIN
5     DECLARE MYNUM1 INT DEFAULT 50;
6     DECLARE MYNUM2 INT DEFAULT 10;
7     DECLARE TOTAL INT;
8     DECLARE MESSAGE VARCHAR(50);
9     SET TOTAL = MYNUM1 * MYNUM2;
10    IF TOTAL < 100 THEN
11        SET MESSAGE = 'The total is small: ';
12    ELSEIF TOTAL >= 100 AND TOTAL <= 500 THEN
13        SET MESSAGE = 'The total is medium: ';
14    ELSEIF TOTAL BETWEEN 500 AND 1000 THEN
15        SET MESSAGE = 'The total is big: ';
16    ELSE
17        SET MESSAGE = 'The total is huge: ';
18    END IF;
19    SELECT CONCAT(MESSAGE, TOTAL) AS 'Result';
20 END;
21 $$
22 delimiter ;
23 • call temp_proc();
```

<

Result Grid | Filter Rows: | Export: | Wrap Cell Contents: |

Result
The total is medium: 500



For Loops

- Loops need an indicator as to when the loop should end and program control should continue through the remainder of the program.
- The first type of loop uses the LOOP keyword to mark the beginning of the loop.
- Loops in MySQL often have a label to name the loop.
- When using the LOOP keyword, a loop label is required.
- The label ends with a colon and precedes the LOOP keyword
- The END LOOP statement marks the end of the loop.
- When logical program control reaches the END LOOP statement, control automatically passes back to the beginning of the loop.
- The **LEAVE** statement is used to exit the loop. Exiting the loop means that logical program control passes immediately to the statement that follows the END LOOP statement.
- The LEAVE statement is typically included as a conditional expression



```
1 • DROP PROCEDURE IF EXISTS temp_proc;
2   delimiter $$
3 • CREATE PROCEDURE TEMP_PROC()
4   BEGIN
5       DECLARE MYNUM INT DEFAULT 1;
6       DECLARE RESULT VARCHAR(100) DEFAULT MYNUM;
7       COUNTER: LOOP
8           SET MYNUM = MYNUM + 1;
9           IF MYNUM > 20 THEN
10              LEAVE COUNTER;
11          END IF;
12          SET RESULT = CONCAT(RESULT, ', ', MYNUM);
13      END LOOP;
14      SELECT RESULT;
15  END;
16  $$
17  delimiter ;
18 • call temp_proc();
```

<	
Result Grid	Filter Rows:
Export:	Wrap Cell Content: ☐
RESULT	
▶ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20	



While Loops

- The second type of loop to consider is called a while loop
- With a WHILE loop, the loop label is optional.
- WHILE loops place the exiting condition immediately after the WHILE keyword
- thus eliminating the need for a LEAVE statement.
-



```
1 • DROP PROCEDURE IF EXISTS temp_proc;
2   delimiter $$
3 • CREATE PROCEDURE TEMP_PROC()
4   BEGIN
5       DECLARE MYNUM INT DEFAULT 1;
6       DECLARE RESULT VARCHAR(100) DEFAULT MYNUM;
7       WHILE MYNUM < 20 DO
8           SET MYNUM = MYNUM + 1;
9           SET RESULT = CONCAT(RESULT, ', ', MYNUM);
10      END WHILE;
11      SELECT RESULT;
12  END;
13  $$
14  delimiter ;
15 • call temp_proc();
```

<	
Result Grid	Filter Rows:
Export:  Wrap Cell Content: 	
RESULT	
▶ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20	



```
1 • DROP PROCEDURE IF EXISTS temp_proc;
2   delimiter $$
3 • CREATE PROCEDURE TEMP_PROC()
4   BEGIN
5       DECLARE MYNUM INT DEFAULT 1;
6       WHILE MYNUM < 20 DO
7           SET MYNUM = MYNUM + 1;
8           SELECT CONCAT('The current value is: ', MYNUM) AS RESULT;
9       END WHILE;
10  END;
11  $$
12  delimiter ;
13 • call temp_proc();
```

<	
Result Grid	Filter Rows:
Export:  Wrap Cell Content: 	
RESULT	
▶ The current value is: 20	



```
1 • DROP PROCEDURE IF EXISTS temp_proc;
2   delimiter $$
3 • CREATE PROCEDURE TEMP_PROC()
4   BEGIN
5       DECLARE MYNUM INT DEFAULT 1;
6       CREATE TEMPORARY TABLE MYTAB(RESULT VARCHAR(100));
7       WHILE MYNUM <= 5 DO
8           INSERT INTO MYTAB VALUES (CONCAT('The current value is ', MYNUM));
9           SET MYNUM = MYNUM + 1;
10      END WHILE;
11      SELECT * FROM MYTAB;
12      DROP TEMPORARY TABLE MYTAB;
13  END;
14  $$
15  delimiter ;
16 • call temp_proc();
```

Result Grid | Filter Rows: | Export:  | Wrap Cell Content: 

RESULT
The current value is 1
The current value is 2
The current value is 3
The current value is 4
The current value is 5



Select Processing with Cursors

- Until now, all of the SELECT statements you have used inside a stored procedure have returned data for output.
- You often need to retrieve data from the database to manipulate within the program.
- In this case, the results of the SELECT query must go into one or more variables so they can be manipulated.



```
1 • DROP PROCEDURE IF EXISTS temp_proc;
2 delimiter $$
3 • CREATE PROCEDURE TEMP_PROC()
4 BEGIN
5     DECLARE W_P1 INT DEFAULT 0;
6     DECLARE W_P2 INT DEFAULT 10;
7     DECLARE W_NUM INT DEFAULT 0;
8     CREATE TEMPORARY TABLE MYTAB(RESULT VARCHAR(100));
9     WHILE W_P2 < 300 DO
10         SELECT COUNT(P_CODE) INTO W_NUM FROM PRODUCT
11         WHERE P_PRICE BETWEEN W_P1 AND W_P2;
12         INSERT INTO MYTAB VALUES (CONCAT('There are ', W_NUM, ' products with a price between '
13         SET W_P1 = W_P2 + 1;
14         SET W_P2 = W_P2 + 50;
15     END WHILE;
16     SELECT * FROM MYTAB;
17     DROP TEMPORARY TABLE MYTAB;
18 END;
19 $$
20 delimiter ;
21 • call temp_proc();
```



PL/SQL Processing with Cursors (1 of 3)

- Cursor: special construct used to hold data rows returned by a SQL query
 - Implicit cursor: automatically created when SQL statement returns only one value
 - Explicit cursor: holds the output of a SQL statement that may return two or more rows
 - Syntax:

`CURSOR cursor_name IS select-query;`

- Cursor-style processing involves retrieving data from the cursor one row at a time
 - Current row is copied to PL/SQL variables



PL/SQL Processing with Cursors (2 of 3)

Cursor Processing Commands	
Cursor Command	Explanation
OPEN	Opening the cursor executes the SQL command and populates the cursor with data, opening the cursor for processing. The cursor declaration command only reserves a named memory area for the cursor; it does not populate the cursor with the data. Before you can use a cursor, you need to open it. For example: <pre>OPEN cursor_name</pre>
FETCH	Once the cursor is opened, you can use the FETCH command to retrieve data from the cursor and copy it to the PL/SQL variables for processing. The syntax is: <pre>FETCH cursor_name INTO variable1 [, variable2, ...]</pre> The PL/SQL variables used to hold the data must be declared in the DECLARE section and must have data types compatible with the columns retrieved by the SQL command. If the cursor's SQL statement returns five columns, there must be five PL/SQL variables to receive the data from the cursor. This type of processing resembles the one-record-at-a-time processing used in previous database models. The first time you fetch a row from the cursor, the first row of data from the cursor is copied to the PL/SQL variables; the second time you fetch a row from the cursor, the second row of data is placed in the PL/SQL variables; and so on.
CLOSE	The CLOSE command closes the cursor for processing.



PL/SQL Processing with Cursors (3 of 3)

Cursor Attributes	
Attribute	Description
%ROWCOUNT	Returns the number of rows fetched so far. If the cursor is not OPEN, it returns an error. If no FETCH has been done but the cursor is OPEN, it returns 0.
%FOUND	Returns TRUE if the last FETCH returned a row, and FALSE if not. If the cursor is not OPEN, it returns an error. If no FETCH has been done, it contains NULL.
%NOT FOUND	Returns TRUE if the last FETCH did not return any row, and FALSE if it did. If the cursor is not OPEN, it returns an error. If no FETCH has been done, it contains NULL.
%ISOPEN	Returns TRUE if the cursor is open (ready for processing) or FALSE if the cursor is closed. Remember, before you can use a cursor, you must open it.



- How do you know when you have reached the end of the cursor data set?
- In MySQL, you must create a special object known as a handler that will signal the end of the cursor has been reached.
- The basic format for a handler is

DECLARE [CONTINUE | EXIT] **HANDLER FOR NOT FOUND** <statement>;

- Whenever the program attempts to FETCH a row from the cursor after there are no rows left to process, the program will check with the handler to determine what action to take.
- The CONTINUE action means that the handler will let the program continue to execute, while the EXIT action means that the program will immediately end.
- If the program is allowed to CONTINUE, then the <statement> will be executed and the program will continue processing at the point at which it realized the cursor was finished.



```
1 • DROP PROCEDURE IF EXISTS temp_proc;
2   delimiter $$
3 • CREATE PROCEDURE TEMP_PROC()
4   BEGIN
5       DECLARE W_P_CODE VARCHAR(15);
6       DECLARE W_P_DESCRIPT VARCHAR(50);
7       DECLARE W_TOT INT DEFAULT 0;
8       DECLARE NOTFOUND INT DEFAULT 0;
9       DECLARE PROD_CURSOR CURSOR FOR SELECT P_CODE, P_DESCRIPT
10                                     FROM PRODUCT
11                                     WHERE P_QOH > (SELECT AVG(P_QOH) FROM PRODUCT);
12   DECLARE CONTINUE HANDLER FOR NOT FOUND SET NOTFOUND = 1;
13   DROP TEMPORARY TABLE IF EXISTS MYTAB;
14   CREATE TEMPORARY TABLE MYTAB(RESULT VARCHAR(100));
15   INSERT INTO MYTAB VALUES ('=====');
16   OPEN PROD_CURSOR;
17   GET_PRODUCTS: LOOP
18       FETCH PROD_CURSOR INTO W_P_CODE, W_P_DESCRIPT;
19       IF NOTFOUND = 1 THEN
20           LEAVE GET_PRODUCTS;
21       END IF;
22       INSERT INTO MYTAB VALUES (CONCAT(W_P_CODE, ' -> ', W_P_DESCRIPT));
23       SET W_TOT = W_TOT + 1;
24   END LOOP;
25   CLOSE PROD_CURSOR;
26   INSERT INTO MYTAB VALUES ('=====');
27   INSERT INTO MYTAB VALUES (CONCAT('TOTAL PRODUCTS PROCESSED: ', W_TOT));
28   INSERT INTO MYTAB VALUES ('--- END OF REPORT ---');
29   SELECT RESULT AS "PRODUCTS WITH P_QOH > AVG(P_QOH)" FROM MYTAB;
30   DROP TEMPORARY TABLE MYTAB;
31   END;
32   $$
33   delimiter ;
34 • call temp_proc();
```



Stored Procedures w/Parameters

```
1 • DROP PROCEDURE IF EXISTS prc_prod_discount;
2   delimiter $$
3 • CREATE PROCEDURE PRC_PROD_DISCOUNT(IN WPI DECIMAL(2,2))
4   BEGIN
5     IF WPI <= 0 OR WPI >= 1 THEN -- validate WPI parameter
6       SELECT 'Error: Value must be greater than 0 and less than 1' AS RESULT;
7     ELSE
8       UPDATE PRODUCT
9       SET P_DISCOUNT = P_DISCOUNT + WPI
10      WHERE P_QOH >= P_MIN * 2;
11      SELECT '** Update finished.';
12    END IF;
13  END;
14  $$
15  delimiter ;
16
17 • START TRANSACTION;
18 • call PRC_PROD_DISCOUNT(.33);
19 • SELECT P_CODE, P_DESCRIPT, P_QOH, P_MIN, P_DISCOUNT FROM PRODUCT;
20 • ROLLBACK;
```



User Defined Functions

- You can also create your own stored functions.
- Stored procedures and functions are very similar.
- A named group of procedural and SQL statements that returns a value, as indicated by a RETURN statement in its program code.

```
CREATE FUNCTION function_name (IN argument data-type, ...)
RETURNS data-type

BEGIN

    Procedure SQL statements;

    ...

    RETURN (value or expression);

END;
```



```
CREATE FUNCTION CUS_NAME(TEMP_CUS_CODE INTEGER)
RETURNS VARCHAR(101) DETERMINISTIC
BEGIN
    DECLARE FNAME VARCHAR(50);
    DECLARE LNAME VARCHAR(50);
    DECLARE INIT VARCHAR(1);

    SELECT CUS_FNAME, CUS_LNAME, CUS_INITIAL
        INTO FNAME, LNAME, INIT
        FROM CUSTOMER
        WHERE CUS_CODE = TEMP_CUS_CODE;

    RETURN CONCAT(FNAME, ' ', LNAME, ' ', COALESCE(CONCAT(INIT, '.', ' ')));
END;
```



Triggers (1 of 5)

- A trigger is procedural SQL code that is automatically invoked by the RDBMS upon the occurrence of a given data manipulation event. It is useful to remember that:
- A trigger is invoked before or after a data row is inserted, updated, or deleted.
- A trigger is associated with a database table.
- Each database table may have one or more triggers.
- A trigger is executed as part of the transaction that triggered it.



Triggers (2 of 5)

- Triggers are critical to proper database operation and management. For example:
- Triggers can be used to enforce constraints that cannot be enforced at the DBMS design and implementation levels.
- Triggers add functionality by automating critical actions and providing appropriate warnings and suggestions for remedial action. In fact, one of the most common uses for triggers is to facilitate the enforcement of referential integrity.
- Triggers can be used to update table values, insert records in tables, and call other stored procedures.
- Triggers cannot output data to the screen. Because triggers are designed to run unattended, the DBMS will not assume that there is any user interface capable of accepting display messages or results of SELECT queries on the user interface.



Triggers (3 of 5)

- Procedural SQL code automatically invoked by RDBMS when given data manipulation event occurs
- Parts of a trigger definition
 - Triggering timing: indicates when trigger's PL/SQL code executes
 - Triggering event: statement that causes the trigger to execute
 - Triggering level: statement- and row-level
 - Triggering action: PL/SQL code enclosed between the BEGIN and END keywords

```
CREATE OR REPLACE TRIGGER trigger_name

[BEFORE / AFTER] [DELETE / INSERT / UPDATE] ON table_name

FOR EACH ROW

BEGIN

PL/SQL instructions;

...

END;
```



Triggers (4 of 5)

- The triggering timing: BEFORE or AFTER. This timing indicates when the trigger's code executes—in this case, before or after the triggering statement is completed.
- The triggering statement or triggering event: The statement that causes the trigger to execute (INSERT, UPDATE, or DELETE).
- The triggering level: The two types of triggers are statement-level triggers and row-level triggers.
 - Although **MySQL does not** currently support statement-level triggers, other DBMSs do, and such functionality may be added to MySQL in the future.
 - A statement-level trigger is executed once, before or after the triggering statement is completed.
 - All triggers are FOR EACH ROW (require for MySQL);
that is, the trigger is activated for each row that is inserted, updated, or deleted



Triggers (5 of 5)

- A row-level trigger requires use of the FOR EACH ROW keywords. This type of trigger is executed once for each row affected by the triggering statement.
 - In other words, if you update 10 rows, the trigger executes 10 times.
- The triggering action: The SQL and procedural SQL code enclosed between the BEGIN and END keywords. Each statement inside the triggering action must end with a semicolon (;).
- One complication with triggers is that you will be creating a situation in which multiple SQL commands are trying to operate on the same rows of data at the same time.
- The trigger timing, BEFORE or AFTER, addresses this to some extent but does not deal with it completely.

```
1 • DROP PROCEDURE IF EXISTS PRC_PRODUCT_REORDER;  
2 delimiter $$  
3 • CREATE PROCEDURE PRC_PRODUCT_REORDER (IN PQOH INT, IN PMIN INT, OUT PREORDER INT)  
4 BEGIN  
5     IF PQOH <= PMIN THEN  
6         SET PREORDER = 1;  
7     ELSE  
8         SET PREORDER = 0;  
9     END IF;  
10 END;  
11 $$  
12 delimiter ;
```



```
1 • DROP TRIGGER IF EXISTS TRG_INSERT_PRODUCT_REORDER;
2   delimiter $$
3 • CREATE TRIGGER TRG_INSERT_PRODUCT_REORDER
4   BEFORE INSERT ON PRODUCT
5   FOR EACH ROW
6   BEGIN
7       CALL PRC_PRODUCT_REORDER(NEW.P_QOH, NEW.P_MIN, NEW.P_REORDER);
8   END;
9   $$
10  delimiter ;
11
```



```
1 • DROP TRIGGER IF EXISTS TRG_UPDATE_PRODUCT_REORDER;  
2   delimiter $$  
3 • CREATE TRIGGER TRG_UPDATE_PRODUCT_REORDER  
4   BEFORE UPDATE ON PRODUCT  
5   FOR EACH ROW  
6   BEGIN  
7       CALL PRC_PRODUCT_REORDER(NEW.P_QOH, NEW.P_MIN, NEW.P_REORDER);  
8   END;  
9   $$  
10  delimiter ;  
11
```

```
1 • DROP TRIGGER IF EXISTS TRG_LINE_PROD;
2 delimiter $$
3 • CREATE TRIGGER TRG_LINE_PROD
4 AFTER INSERT ON LINE
5 FOR EACH ROW
6 BEGIN
7     UPDATE PRODUCT
8     SET P_QOH = P_QOH - NEW.LINE_UNITS
9     WHERE PRODUCT.P_CODE = NEW.P_CODE;
10 END;
11 $$
12 delimiter ;
13
```

```

1 • DROP TRIGGER IF EXISTS TRG_LINE_CUS;
2 delimiter $$
3 • CREATE TRIGGER TRG_LINE_CUS
4 AFTER INSERT ON LINE
5 FOR EACH ROW
6 BEGIN
7     DECLARE W_CUS CHAR(5);
8     DECLARE W_TOT DECIMAL(10,2) DEFAULT 0; -- to compute total cost
9     -- this trigger fires up after an INSERT of a LINE
10    -- it will update the CUS_BALANCE in CUSTOMER
11
12    -- 1) get the CUS_CODE
13    SELECT CUS_CODE INTO W_CUS
14    FROM INVOICE
15    WHERE INVOICE.INV_NUMBER = NEW.INV_NUMBER;
16
17    -- 2) compute the total for the current line
18    SET W_TOT = NEW.LINE_PRICE * NEW.LINE_UNITS;
19
20    -- 3) update the CUS_BALANCE in CUSTOMER
21    UPDATE CUSTOMER
22    SET CUS_BALANCE = CUS_BALANCE + W_TOT
23    WHERE CUS_CODE = W_CUS;
24 END;
25 $$
26 delimiter ;

```



Deleting Triggers

- DROP TRIGGER trigger_name command
- *DROP TRIGGER IF EXISTS....*
 - Deletes a trigger without deleting the table
- Trigger action based on conditional DML predicates
 - Actions depend on the type of DML statement that fires the trigger



Embedded SQL (1 of 4)

- SQL statements contained within an application programming language
 - Host language: any language that contains embedded SQL statements
- Differences between SQL and procedural languages
 - Run-time mismatch
 - SQL is executed one instruction at a time
 - Host language runs at client side in its own memory space
 - Processing mismatch
 - Conventional programming languages process one data element at a time
 - Newer programming environments manipulate data sets in a cohesive manner
 - Data type mismatch
 - Data types provided by SQL might not match data types used in different host languages



Embedded SQL (2 of 4)

- Embedded SQL framework defines:
 - Standard syntax to identify embedded SQL code within the host language
 - Standard syntax to identify host variables
 - Communication area used to exchange status and error information between SQL and host language



Embedded SQL (3 of 4)

SQL Status and Error Reporting Variables		
Variable Name	Value	Explanation
SQLCODE		Old-style error reporting supported for backward compatibility only; returns an integer value (positive or negative)
	0	Successful completion of command
	100	No data; the SQL statement did not return any rows and did not select, update, or delete any rows
	-999	Any negative value indicates that an error occurred
SQLSTATE		Added by SQL-92 standard to provide predefined error codes; defined as a character string (5 characters long)
	00000	Successful completion of command
		Multiple values in the format XXYYY where: XX-> represents the class code YYY-> represents the subclass code



Embedded SQL (4 of 4)

- Static SQL: programmer uses predefined SQL statements and parameters
 - SQL statements will not change while application is running
- Dynamic SQL: SQL statement is generated at run time
 - Attribute list and condition are not known until end user specifies them
 - Slower than static SQL
 - Requires more computer resources
 - Inconsistent levels of support and incompatibilities among DBMS vendors



Summary (1 of 2)

- The ANSI standard data types are supported by all RDBMS vendors in different ways
- The basic data definition commands allow you to create tables and indexes
- Data manipulation commands allow you to add, modify, and delete rows from tables
- Views can be created to expose subsets of data to end users primarily for security and privacy reasons
- In Oracle and SQL Server, sequences may be used to generate values to be assigned to a record
- Procedural Language SQL (PL/SQL) can be used to create triggers, stored procedures, and PL/SQL functions



Summary (2 of 2)

- A stored procedure is a named collection of SQL statements
- When SQL statements are designed to return more than one value inside the PL/SQL code, a cursor is needed
- Embedded SQL refers to the use of SQL statements within an application programming language such as Visual Basic .NET, C#, COBOL, or Java